Async Credit Processing Service

1. Problem and scope

A client submits a credit and receives 202 Accepted with a task ID; a worker updates an existing user's balance later. The design prioritizes durably recorded requests, at-most-once balance effects and exact decimal amounts. RabbitMQ can redeliver messages; correctness is guaranteed by MongoDB transactions.

Assumptions: Existing users only (404 otherwise); single currency; positive amounts up to 1,000,000.00 with at most two decimal places. Demo API keys identify clients; account-level authorization is out of scope.

2. Architecture and lifecycle

Client	FastAPI	RabbitMQ	Celery worker
POST /credits	save operation; publish ID	durable delivery	commit balance + status

1 / Accept. FastAPI validates identity, money and overload limits. A transaction inserts a processing operation and increments the user's pending count. The API publishes the operation ID (not the amount) and returns 202; if RabbitMQ is unavailable, the durable MongoDB record remains recoverable.

2 / Apply. Celery loads the operation, conditionally transitions it to processed, updates the balance and decrements the pending count in one transaction. It acknowledges after commit. Group commit may settle up to 100 eligible credits for one user in one transaction.

3 / Recover. Celery Beat schedules the dispatcher every 15s. Token-based, expiring leases coordinate bounded publication batches. Never-confirmed credits are retried; previously confirmed but unsettled credits become eligible after 15 minutes only when the queue is nearly empty, avoiding replay storms.

Components: FastAPI (HTTP); MongoDB 8.2 replica set (source of truth); RabbitMQ (durable/persistent messaging with publisher confirms); Celery (late ACKs and retries); Beat (periodic dispatcher). API, worker and Beat share one codebase with framework-independent business services.

3. Data model and correctness

• users: unique user ID, Decimal128 balance, aggregate status and pending_credits.

• credit_operations: ID, client/user, Decimal128 amount, authoritative status, scoped idempotency key and fingerprint, timestamps, attempts/crashes and dispatch claims. The same document acts as the outbox.

• Exact money: Python Decimal → BSON Decimal128 → JSON decimal strings. An operation-level status describes multiple credits independently; the user status is aggregate.

Key invariant: At-least-once message delivery, but at-most-once balance effect. A guarded processing-to-processed transition and the balance increment commit together; repeated messages are no-ops. Per-client idempotency keys also protect HTTP retries.

Reliability, validation and trade-offs

4. Failure and overload handling

Failure scenario	Response
Worker crashes after commit	RabbitMQ redelivers; the settled credit is skipped and ACKed.
Broker unavailable / lost publish	Dispatcher republishes durable operations from MongoDB.
Transient DB error / uncertain commit	Retry or re-read persisted state; infrastructure failure never automatically fails an accepted credit.
Repeated worker crashes	After 10, park the owed credit, alert an operator and require explicit unpark.
User deleted before settlement	Non-recoverable business failure; do not change the balance.
Excessive outstanding work	Reject before saving: 429 at 1,000 pending/user; 503 for broker backlog.

Durability and deadlines: Explicit majority-journaled MongoDB writes and snapshot transaction reads; publisher confirms, persistent messages and late consumer acknowledgments. The API caps total MongoDB time at 10s; worker transaction retries may run longer. JSON logs, health probes, metrics and tested Prometheus alert rules support operations.

5. Testing and verification

94 automated tests cover amount precision, idempotency races, concurrency, guarded status transitions, group commit, dispatcher leases, API limits, task redelivery and end-to-end processing. Integration tests use real MongoDB and RabbitMQ. Failure injection targets commit-reply loss and process crashes at publish/commit/ACK boundaries.

Checked invariants during concurrent execution: balance = opening balance + sum of processed credits; pending count = unsettled operations. Breaking correctness guards intentionally caused corresponding tests to fail.

Real-stack drills: 2,000 requests while RabbitMQ was down; 500 queued messages surviving broker restart; repeated SIGKILL during 8,000 credits; MongoDB unavailable for 30s and 150s; and 3,000 queued messages without a republish storm. Local CI-equivalent checks include Ruff, strict MyPy, tests, alert-rule checks, build and smoke test.

6. Trade-offs and production path

• Deployment: Docker Compose with single-node MongoDB/RabbitMQ is reproducible but not HA. Production needs a multi-node replica set, RabbitMQ quorum queues, multiple API/workers, TLS, stronger identity/authorization, managed secrets, backups and failover drills.

• Capacity: Group commit improved measured hot-account settlement from 76 to 145 credits/s locally. Submission still writes the user record per accepted request; ~300 overall requests/s is a laptop observation, not a production guarantee.

• Operations: Parked credits require human ownership. Production also needs live alert routing, reconciliation of balances to operations, retention/archival policy and sustained soak testing.